Perform Bias/Risk Assesment of Model

In [1]:
```
import numpy as np
import pandas as pd
%pip install shap
import yfinance as yf
import warnings
import matplotlib.pyplot as plt
import seaborn as sns
from pathlib import Path
import zipfile
import requests
warnings.filterwarnings("ignore")
```

```
Defaulting to user installation because normal site-packages is not wri
teable
Requirement already satisfied: shap in ./.local/lib/python3.12/site-pac
kages (0.51.0)
Requirement already satisfied: numpy>=2 in /opt/tljh/user/lib/python3.1
2/site-packages (from shap) (2.2.6)
Requirement already satisfied: scipy in /opt/tljh/user/lib/python3.12/s
ite-packages (from shap) (1.17.0)
Requirement already satisfied: scikit-learn in /opt/tljh/user/lib/pytho
n3.12/site-packages (from shap) (1.8.0)
Requirement already satisfied: pandas in /opt/tljh/user/lib/python3.12/
site-packages (from shap) (2.3.3)
Requirement already satisfied: tqdm>=4.27.0 in /opt/tljh/user/lib/pytho
n3.12/site-packages (from shap) (4.66.5)
Requirement already satisfied: packaging>20.9 in /opt/tljh/user/lib/pyt
hon3.12/site-packages (from shap) (24.1)
Requirement already satisfied: slicer==0.0.8 in ./.local/lib/python3.1
2/site-packages (from shap) (0.0.8)
Requirement already satisfied: numba in ./.local/lib/python3.12/site-pa
ckages (from shap) (0.65.0)
Requirement already satisfied: llvmlite in ./.local/lib/python3.12/site
-packages (from shap) (0.47.0)
Requirement already satisfied: cloudpickle in ./.local/lib/python3.12/s
ite-packages (from shap) (3.1.2)
Requirement already satisfied: typing-extensions in /opt/tljh/user/lib/
python3.12/site-packages (from shap) (4.15.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /opt/tljh/use
r/lib/python3.12/site-packages (from pandas->shap) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /opt/tljh/user/lib/pytho
n3.12/site-packages (from pandas->shap) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /opt/tljh/user/lib/pyt
hon3.12/site-packages (from pandas->shap) (2025.3)
Requirement already satisfied: joblib>=1.3.0 in /opt/tljh/user/lib/pyth
on3.12/site-packages (from scikit-learn->shap) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.2.0 in /opt/tljh/user/l
ib/python3.12/site-packages (from scikit-learn->shap) (3.6.0)
Requirement already satisfied: six>=1.5 in /opt/tljh/user/lib/python3.1
2/site-packages (from python-dateutil>=2.8.2->pandas->shap) (1.17.0)
Note: you may need to restart the kernel to use updated packages.
```

In [2]:

```python
import os, subprocess

REPO_URL = "https://github.com/tongyuguo/HelpHerInvest.git"
REPO_DIR = "HelpHerInvest"
data_dir = os.path.join(REPO_DIR, "Data")

def clone_or_pull():
    if os.path.isdir(os.path.join(REPO_DIR, ".git")):
        subprocess.run(["git", "-C", REPO_DIR, "pull"])
    else:
        subprocess.run(["git", "clone", REPO_URL])

clone_or_pull()
```

```
From https://github.com/tongyuguo/HelpHerInvest
   774b74f..b804b87  main       -> origin/main
Updating 774b74f..b804b87
Fast-forward
 Code Outputs/Week 10/week10_data_centric_model.pdf |  Bin 0 -> 213684
bytes
 Notebooks/Week 10/week10_data_centric_model.ipynb  | 4254 ++++++++++++
++++++++
 .../week11_bias_assessment_modell01.ipynb          | 1741 ++++++++
 Reports/Week 10/test.html                          |    1 +
 .../week10_helpherinvest_written_report.pdf        |  Bin 0 -> 421770
bytes
 .../week11_helpherinvest_written_report.pdf        |  Bin 0 -> 222349
bytes
 6 files changed, 5996 insertions(+)
 create mode 100644 Code Outputs/Week 10/week10_data_centric_model.pdf
 create mode 100644 Notebooks/Week 10/week10_data_centric_model.ipynb
 create mode 100644 Playground/Week 11 Explain the model, analyze risk,
bias and ethical considerations/Chris Playground/week11_bias_assessment
_modell01.ipynb
 create mode 100644 Reports/Week 10/test.html
 create mode 100644 Reports/Week 10/week10_helpherinvest_written_repor
t.pdf
 create mode 100644 Reports/Week 11/week11_helpherinvest_written_repor
t.pdf
```

In [3]:

```python
# Load the data from the zip file

## PATH NEEDS TO BE UPDATED TO GITHUB DATA PATH
zip_path2 = "HelpHerInvest/Data/final_dataset_20260224v2.csv.zip"
with zipfile.ZipFile(zip_path2) as z:
    df = pd.read_csv(z.open("final_dataset_20260224v2.csv"))

## ## ## ##

tickers = df["Ticker"].unique()
print(f"Unique tickers in final dataset: {len(tickers)}")
print("Size of dataset:",
"Rows:",df.shape[0],
"Columns:",df.shape[1])
```

```
Unique tickers in final dataset: 1993
Size of dataset: Rows: 302024 Columns: 18
```

In [4]:

```python
# count number of positive fwd_excess_return
print("Number of positive fwd_excess_return:", (df["fwd_excess"] > 0).
print("Number of negative fwd_excess_return:", (df["fwd_excess"] < 0).
print("Number of zero fwd_excess_return:", (df["fwd_excess"] == 0).sum

df_temp = df.copy()

df_temp['y'] = (df_temp['fwd_excess'] > 0).astype(int)
```

```
Number of positive fwd_excess_return: 143948
Number of negative fwd_excess_return: 151730
Number of zero fwd_excess_return: 287
```

In [5]:
```
df = df_temp.dropna()
print("---  ---  ---")
print("Length of original:",len(df_temp.index))
print("Length of dropped:",len(df.index))
```

```
---  ---  ---
Length of original: 302024
Length of dropped: 274511
```

Create Winning Dataset - Cross Sectionally Ranked

In [6]:
```
rank_cols = [
    'mom_1m','mom_3m','mom_6m','mom_12m','mom_12m_ex_1m',
    'rel_3m_spy','rel_6m_spy','rel_12m_spy',
    'vol_3m','vol_6m',
    'drawdown_6m','drawdown_12m',
    'pct_above_200dma'
]

df_rank = df.copy()

for col in rank_cols:
    df_rank[col] = df_rank.groupby("Date")[col].rank(pct=True)

#df_rank[rank_cols].head(10)
```

In [7]:
```
df_rank["fwd_rank"] = df_rank.groupby("Date")["fwd_excess"].rank(pct=T

df_rank["target"] = pd.cut(
    df_rank["fwd_rank"],
    bins=[0, .2, .4, .6, .8, 1],
    labels=[0,1,2,3,4]
)

df_rank.head(10)
```

Out[7]:

	Date	Ticker	mom_1m	mom_3m	mom_6m	mom_12m	mom_12m_ex
13389	2011-01-31	NVDA	0.997496	0.995826	0.995826	0.814691	0.137
13390	2011-01-31	GOOGL	0.480801	0.142738	0.636060	0.281302	0.276
13391	2011-01-31	AAPL	0.729549	0.623539	0.766277	0.909015	0.889
13392	2011-01-31	MSFT	0.381469	0.343907	0.277963	0.121870	0.144
13393	2011-01-31	AMZN	0.152755	0.280467	0.883973	0.601836	0.727
13394	2011-01-31	AVGO	0.479967	0.727045	0.768781	0.868114	0.873
13396	2011-01-31	BRK-B	0.545910	0.282137	0.193656	0.191987	0.180
13397	2011-01-31	LLY	0.373122	0.210351	0.120200	0.154424	0.184
13398	2011-01-31	WMT	0.665275	0.327212	0.330551	0.196160	0.166
13399	2011-01-31	JPM	0.760434	0.785476	0.361436	0.322204	0.227

10 rows × 21 columns

In [8]:

```
def time_split(
    df,
    date_col="Date",
    train_frac=0.70,
    val_frac=0.15,
    test_frac=0.15,
    sort_dates=True
):
    if abs(train_frac + val_frac + test_frac - 1.0) > 1e-8:
        raise ValueError("train_frac + val_frac + test_frac must equal
```

```
    data = df.copy()
    data[date_col] = pd.to_datetime(data[date_col])

    if sort_dates:
        data = data.sort_values(date_col)

    unique_dates = pd.Series(data[date_col].drop_duplicates()).sort_va

    n_dates = len(unique_dates)
    train_end = int(n_dates * train_frac)
    val_end = int(n_dates * (train_frac + val_frac))

    train_dates = unique_dates.iloc[:train_end]
    val_dates = unique_dates.iloc[train_end:val_end]
    test_dates = unique_dates.iloc[val_end:]

    train_df = data[data[date_col].isin(train_dates)].copy()
    val_df = data[data[date_col].isin(val_dates)].copy()
    test_df = data[data[date_col].isin(test_dates)].copy()

    return train_df, val_df, test_df
```

Preprocess the Data

In [9]:
```
def preprocess_data(df):

    # preprocess the data by dropping rows with missing values
    print("Missing fwd_excess values:")

    df_dropped = df.dropna()
    print("---  ---  ---")
    print("Length of original:",len(df.index))
    print("Length of dropped:",len(df_dropped.index))
    print("Sanity check - missing fwd_excess values after dropping:")
    print(df_dropped[["Date","Ticker","fwd_excess"]].tail(5))



    # Create log_adj_close column
    df_dropped["log_adj_close"] = np.log(df_dropped["adj_close"])
    df_dropped[["adj_close","log_adj_close"]].hist(figsize=(14, 10), b
    plt.tight_layout()
    plt.show()

    return df_dropped
```

In [10]:
```
def retrieve_features(df):
    feature_cols = [col for col in df.columns if col not in ["Date", "
    return feature_cols
```

In [11]:
```
# Dataset 1 - Using Ranked Features
```

```
train_df_rank, val_df_rank, test_df_rank = time_split(df_rank, date_co
X_train_rank = train_df_rank[retrieve_features(train_df_rank)]
y_train_rank = train_df_rank["y"]
X_val_rank = val_df_rank[retrieve_features(val_df_rank)]
y_val_rank = val_df_rank["y"]
X_test_rank = test_df_rank[retrieve_features(test_df_rank)]
y_test_rank = test_df_rank["y"]
```

Light Gradient Boosting Machine (LightGBM) Models

In [12]:

```
# Winning LightGBM Model - Dataset 1

import lightgbm as lgb

lgb_model1 = lgb.LGBMClassifier(
    n_estimators=1000,
    learning_rate=0.01,
    max_depth=12,
    num_leaves=31,
    class_weight="balanced",
)

lgb_model1.fit(
    X_train_rank,
    y_train_rank,
    eval_set=[(X_val_rank, y_val_rank)],

)
```

```
[LightGBM] [Info] Number of positive: 86755, number of negative: 89946
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead
of testing was 0.013261 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 3570
[LightGBM] [Info] Number of data points in the train set: 176701, numbe
r of used features: 14
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=
0.000000
[LightGBM] [Info] Start training from score 0.000000
```

Out[12]:

```
▾ LGBMClassifier ⓘ
▸ Parameters
```

In [13]:

```
from sklearn.metrics import f1_score, classification_report, confusion


## Used to Evaluate the Logistic Regression and LightGBM models ##
def evaluate(model, X_test, y_test):
    y_pred = model.predict(X_test)
```

```
    probs = model.predict_proba(X_test)[:,1]

    print("Classification Report:\n")
    print(classification_report(y_test, y_pred))
    print("F1:", round(f1_score(y_test, y_pred, average='macro'), 4))
    print("ROC-AUC:", round(roc_auc_score(y_test, probs, average='macr


print("LightGBM Model 1:")
evaluate(lgb_model1, X_val_rank, y_val_rank)
print("------------  --------------  ------------  --------------")
```

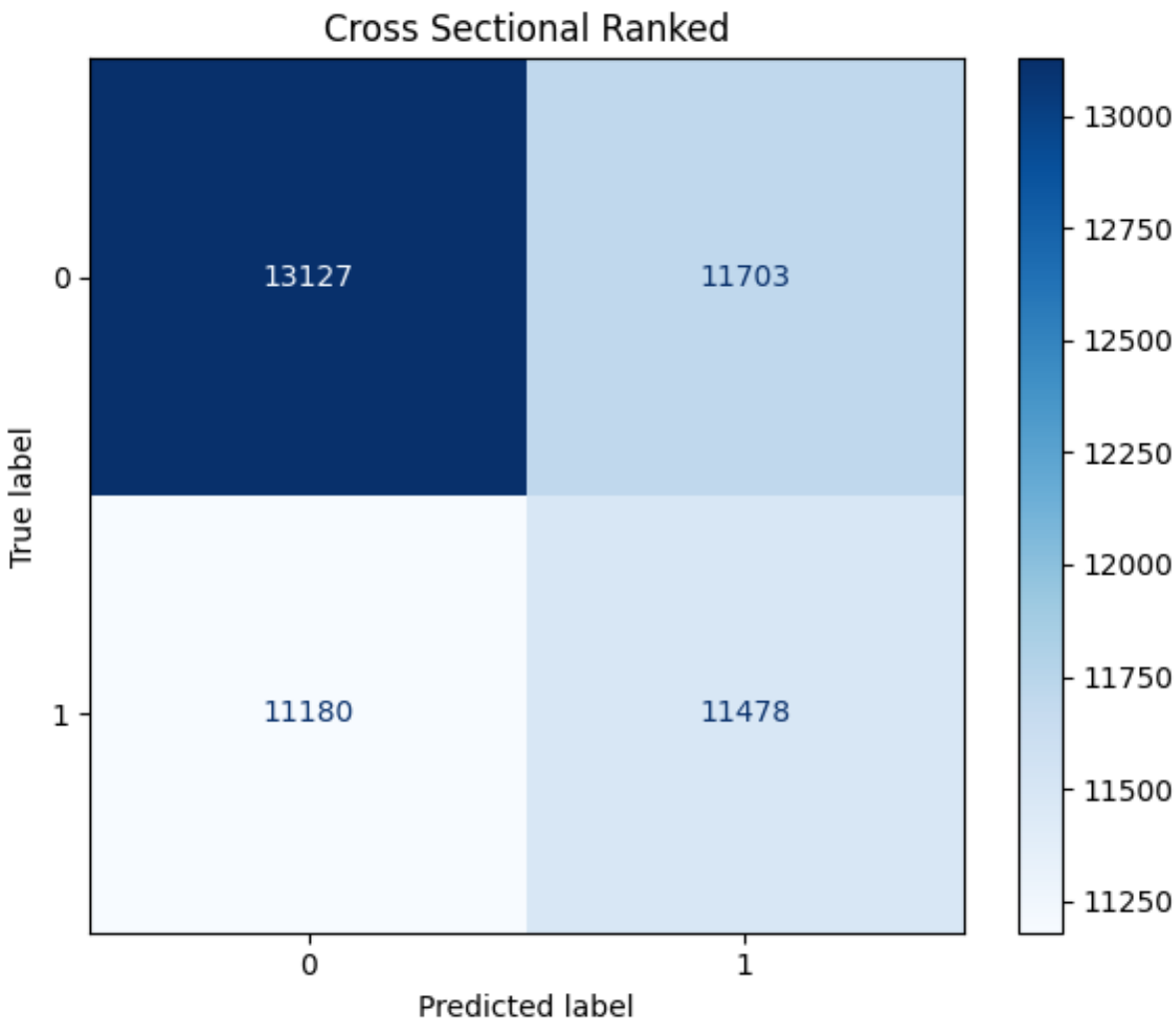
```
LightGBM Model 1:
Classification Report:

              precision    recall  f1-score   support

           0       0.54      0.53      0.53     24830
           1       0.50      0.51      0.50     22658

    accuracy                           0.52     47488
   macro avg       0.52      0.52      0.52     47488
weighted avg       0.52      0.52      0.52     47488

F1: 0.5175
ROC-AUC: 0.5234
------------  --------------  ------------  --------------
```

In [14]:

```
from sklearn.metrics import ConfusionMatrixDisplay
import matplotlib.pyplot as plt

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

def create_confusion_matrix(model, X_val, y_val, name):
    y_pred = model.predict(X_val)

    cm = confusion_matrix(y_val, y_pred)
    cm_df = pd.DataFrame(
        cm,
        index=["Actual 0", "Actual 1"],
        columns=["Pred 0", "Pred 1"]
    )

    print(f"\n{name}")
    print(cm_df)

    fig, ax = plt.subplots(figsize=(6, 5))
    ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=[0, 1])
        ax=ax,
        cmap="Blues",
        values_format="d"
```

```
    )
    ax.set_title(name)
    plt.tight_layout()
    plt.show()

    return cm_df


create_confusion_matrix(lgb_model1, X_val_rank, y_val_rank, "Cross Sec
```

```
Cross Sectional Ranked
          Pred 0  Pred 1
Actual 0   13127   11703
Actual 1   11180   11478
```



Out[14]:

	Pred 0	Pred 1
Actual 0	13127	11703
Actual 1	11180	11478

In [15]:

```
# Selected the best model from validation performance and evaluate on
print("\n-----  -----  -----  -----  -----  -----  -----  -----  -----
print("LightGBM Model 1 - Test Set:")
```

```
evaluate(lgb_model1, X_test_rank, y_test_rank)
```

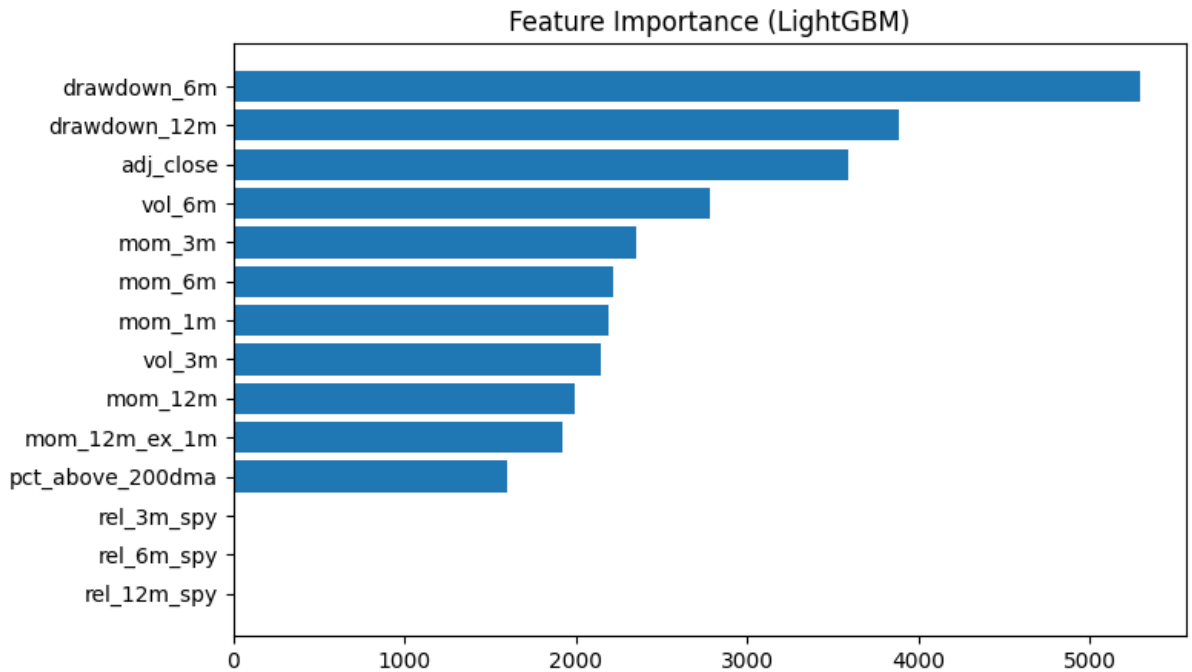
```
-----  -----  -----  -----  -----  -----  -----  -----  -----  -----
LightGBM Model 1 - Test Set:
Classification Report:

              precision    recall  f1-score   support

           0       0.54      0.52      0.53     26761
           1       0.48      0.49      0.49     23561

    accuracy                           0.51     50322
   macro avg       0.51      0.51      0.51     50322
weighted avg       0.51      0.51      0.51     50322

F1: 0.5087
ROC-AUC: 0.5126
```

Determining Important Features

In [16]:

```
importance = pd.DataFrame({
    "feature": X_train_rank.columns,
    "importance": lgb_model1.feature_importances_
}).sort_values(by="importance", ascending=False)

print(importance.head(15))

# Plot
plt.figure(figsize=(8,5))
plt.barh(importance["feature"][:15][::-1], importance["importance"][:1
plt.title("Feature Importance (LightGBM)")
plt.show()
```

```
             feature  importance
10       drawdown_6m        5298
11      drawdown_12m        3889
13         adj_close        3592
9             vol_6m        2783
1             mom_3m        2351
2             mom_6m        2222
0             mom_1m        2194
8             vol_3m        2150
3            mom_12m        1997
4      mom_12m_ex_1m        1926
12  pct_above_200dma        1598
5         rel_3m_spy           0
6         rel_6m_spy           0
7        rel_12m_spy           0
```



In [17]:
```
from sklearn.inspection import permutation_importance

result = permutation_importance(
    lgb_model1, X_val_rank, y_val_rank,
    n_repeats=10,
    random_state=42,
    scoring="f1"
)

perm_importance = pd.DataFrame({
    "feature": X_val_rank.columns,
    "importance": result.importances_mean
}).sort_values(by="importance", ascending=False)

print(perm_importance.head(15))
```

```
             feature  importance
1             mom_3m    0.040786
0             mom_1m    0.019287
3            mom_12m    0.012487
13         adj_close    0.009978
2             mom_6m    0.004466
5         rel_3m_spy    0.000000
7        rel_12m_spy    0.000000
6         rel_6m_spy    0.000000
8             vol_3m   -0.000038
4      mom_12m_ex_1m   -0.001346
9             vol_6m   -0.001655
12  pct_above_200dma   -0.001883
11      drawdown_12m   -0.009322
10       drawdown_6m   -0.034162
```

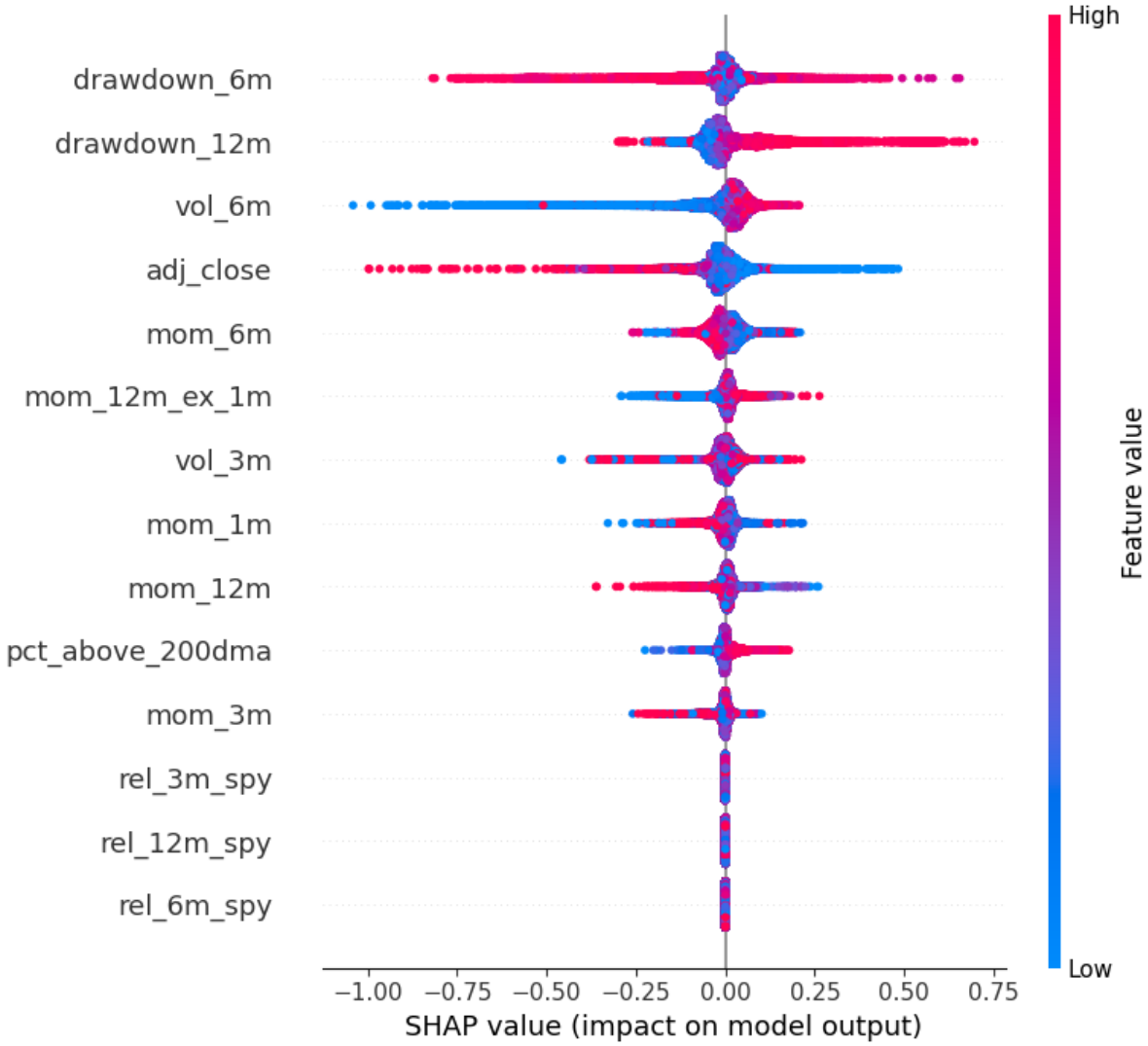
In [18]:
```
#%pip install shap
```

```python
import shap
import pandas as pd
import numpy as np

# Fit explainer (LightGBM example)
explainer = shap.TreeExplainer(lgb_model1)
shap_values = explainer.shap_values(X_val_rank)

# For binary classification, use class 1
shap_vals = shap_values[1] if isinstance(shap_values, list) else shap_

# Mean absolute SHAP values = global importance
importance_df = pd.DataFrame({
    "feature": X_val_rank.columns,
    "importance": np.abs(shap_vals).mean(axis=0)
}).sort_values(by="importance", ascending=False)

top_features = importance_df.head(10)
print(top_features)
```

```
             feature  importance
10       drawdown_6m    0.063273
11      drawdown_12m    0.060172
9             vol_6m    0.048394
13         adj_close    0.035371
2             mom_6m    0.027896
4      mom_12m_ex_1m    0.019576
8             vol_3m    0.018278
0             mom_1m    0.016098
3            mom_12m    0.013657
12  pct_above_200dma    0.012992
```

In [19]:

```python
shap.summary_plot(shap_vals, X_val_rank)
```



In [20]:
```
sample_pos = np.random.choice(len(X_val_rank), size=5, replace=False)
X_sample = X_val_rank.iloc[sample_pos]
shap_sample = shap_vals[sample_pos]
```

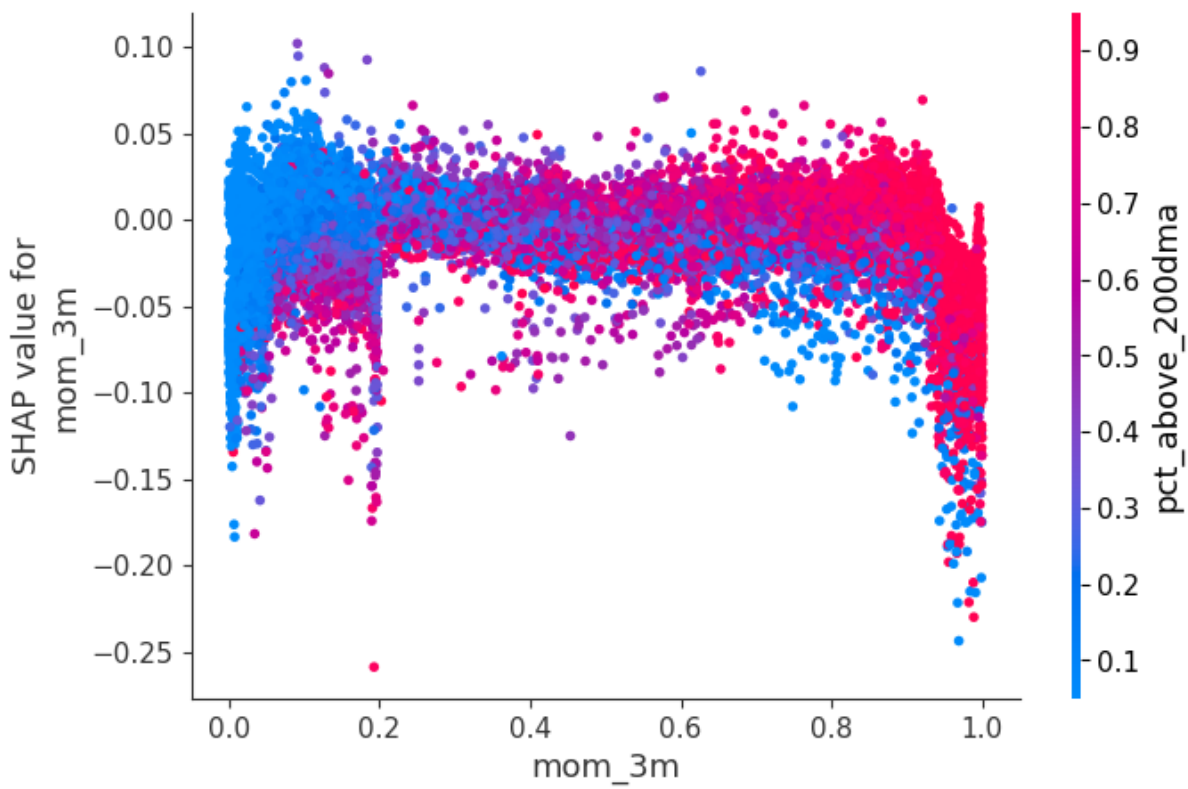
In [21]:
```
import random
random.seed(42)

for i, idx in enumerate(sample_pos):
    print(f"\n--- Prediction {i+1} (Index {idx}) ---")

    row = X_val_rank.iloc[idx]
    shap_row = shap_sample[i]

    # Combine into DataFrame
    explanation = pd.DataFrame({
        "feature": X_val_rank.columns,
        "value": row.values,
        "shap_impact": shap_row
    }).sort_values(by="shap_impact", key=abs, ascending=False)

    print(explanation.head(10))  # Top drivers
```

```
--- Prediction 1 (Index 45698) ---
             feature      value  shap_impact
9             vol_6m   0.081510    -0.061590
0             mom_1m   0.359409     0.043662
12  pct_above_200dma   0.254923    -0.028787
2             mom_6m   0.334792     0.023826
10       drawdown_6m   0.168490     0.022338
11      drawdown_12m   0.289934    -0.018990
4      mom_12m_ex_1m   0.705142     0.010675
1             mom_3m   0.060722    -0.010581
13         adj_close  92.137600    -0.008547
3            mom_12m   0.648796     0.003924

--- Prediction 2 (Index 17185) ---
             feature     value  shap_impact
9             vol_6m  0.002875    -0.665546
8             vol_3m  0.002875    -0.143544
11      drawdown_12m  0.930995     0.075395
10       drawdown_6m  0.914319    -0.070043
13         adj_close  9.890000    -0.067423
2             mom_6m  0.673951    -0.060178
1             mom_3m  0.676251    -0.047705
4      mom_12m_ex_1m  0.449684     0.044313
0             mom_1m  0.833237     0.011210
12  pct_above_200dma  0.700403     0.002092

--- Prediction 3 (Index 41214) ---
             feature      value  shap_impact
9             vol_6m   0.082876    -0.079083
10       drawdown_6m   0.611690     0.063364
3            mom_12m   0.361690     0.050384
13         adj_close  59.345900    -0.046686
4      mom_12m_ex_1m   0.414105     0.014258
8             vol_3m   0.134468     0.013621
0             mom_1m   0.324643     0.011541
12  pct_above_200dma   0.392426    -0.010441
11      drawdown_12m   0.588364    -0.007941
1             mom_3m   0.661910    -0.005008

--- Prediction 4 (Index 13730) ---
             feature     value  shap_impact
9             vol_6m  0.813372     0.100628
10       drawdown_6m  0.566860     0.089345
4      mom_12m_ex_1m  0.153488    -0.031016
13         adj_close  2.910000     0.026984
2             mom_6m  0.890116    -0.026190
1             mom_3m  0.369477    -0.016196
12  pct_above_200dma  0.618605    -0.014046
11      drawdown_12m  0.326163    -0.012114
0             mom_1m  0.952907    -0.009511
8             vol_3m  0.786628     0.004275
```

```
--- Prediction 5 (Index 16341) ---
              feature       value  shap_impact
13          adj_close  353.951000    -0.087460
4       mom_12m_ex_1m    0.565067     0.031887
3             mom_12m    0.606709     0.025380
0              mom_1m    0.625217     0.021654
10        drawdown_6m    0.484962     0.016028
11       drawdown_12m    0.571139     0.010056
12   pct_above_200dma    0.466165    -0.007510
8              vol_3m    0.329092     0.005293
2              mom_6m    0.485830    -0.003432
1              mom_3m    0.393580    -0.001006
```

In [22]:
```
explanation.head(5)
```

Out[22]:

	feature	value	shap_impact
13	adj_close	353.951000	-0.087460
4	mom_12m_ex_1m	0.565067	0.031887
3	mom_12m	0.606709	0.025380
0	mom_1m	0.625217	0.021654
10	drawdown_6m	0.484962	0.016028

In [23]:
```
shap.dependence_plot("mom_3m", shap_vals, X_val_rank)
```